PART-A

****

Program No: 1

Write a program to sort a list of N elements using Selection Sort Technique.

****

```
n = int(input("Enter the number of elements: "))

arr = []

for i in range(n):
    num = int(input("Enter the element: "))
    arr.append(num)

for i in range(n):
    min_index = i

    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

print("The Selection Sorted List is:")
print(arr)
```

OUTPUT:

```
Enter the number of elements: 6
enter the element:8
enter the element:6
enter the element:5
enter the element:4
enter the element:2
enter the element:1
The selection Sorted list is :
[1, 2, 4, 5, 6, 8]
```

****

Program No: 2

Write a program to sort a list of N elements using Insertion Sort Technique.

****

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key
n = int(input("Enter number of elements: "))
arr = []

for i in range(n):
    num = int(input("Enter the elements: "))
    arr.append(num)

print("Array before sorting:", arr)
insertion_sort(arr)
print("Array after sorting:", arr)
```

OUTPUT:

```
Enter number of elements: 5
Enter the elements: 7
Enter the elements: 2
Enter the elements: 9
Enter the elements: 1
Enter the elements: 5
Array before sorting: [7, 2, 9, 1, 5]
Array after sorting: [1, 2, 5, 7, 9]
```

****

Program No: 3

Write a program to read 'n' numbers, find minimum and maximum value in an array using divide and conquer.

****

```
def find_min_max(arr, low, high):
    if low == high:
      return arr[low], arr[low]
    if high == low + 1:
      if arr[low] < arr[high]:
        return arr[low], arr[high]
      else:
        return arr[high], arr[low]
    mid = (low + high) // 2
    min1, max1 = find_min_max(arr, low, mid)
    min2, max2 = find_min_max(arr, mid + 1, high)
    return min(min1, min2), max(max1, max2)
n = int(input("Enter number of elements: "))
arr=[]

for i in range(n):
    num = int(input("Enter the elements: "))
    arr.append(num)
minimum, maximum = find_min_max(arr, 0, n - 1)
print("Minimum element =", minimum)
print("Maximum element =", maximum)
```

OUTPUT:

```
Enter number of elements: 5
Enter the elements: 8
Enter the elements: 2
Enter the elements: 5
Enter the elements: 9
Enter the elements: 4
Minimum element = 2
Maximum element = 9
```

****

Program No: 4

Write program to implement the BFS algorithm for a graph.

****

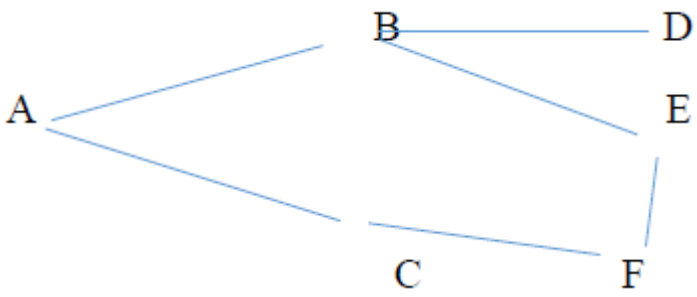
```
from collections import deque
queue=deque()

def bfs(visited, graph,node):
    visited.add(node)
    queue.appendleft(node)
    while queue:
        node =queue.pop()
        print(node,end = "\t")
        for neigbour in graph[node]:
            if neigbour not in visited:
                visited.add(neigbour)
                queue.appendleft(neigbour)

graph = {
'A': ['B', 'C'],
'B': ['A', 'D', 'E'],
'C': ['A', 'F'],
'D': [],
'E': ['B', 'F'],
'F': ['C', 'E']
}

visited=set()
print("Following is the Breadth First Traversal")
bfs(visited, graph, 'A')
```

Graph pictorial representation



OUTPUT:

```
Following is the Breadth First Traversal
A       B       C       D       E       F
```

****

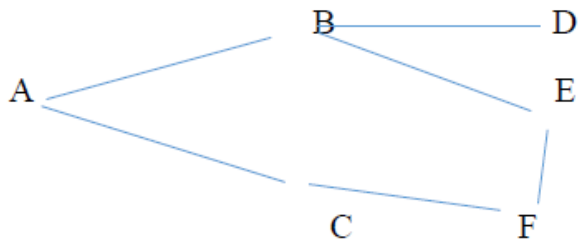
Program No: 5

Write program to implement the DFS algorithm for a graph

****

```
graph = {
'A': ['B', 'C'],
'B': ['A', 'D', 'E'],
'C': ['A', 'F'],
'D': [],
'E': ['B', 'F'],
'F': ['C', 'E']
}
visited = set()
def dfs(visited,graph,node):
    if node not in visited:
        print(node,end='\t')
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited,graph,neighbour)
print("Following is the Depth first search traversal")
dfs(visited, graph, 'A')
```

Graph pictorial representation



OUTPUT:

```
Following is the Depth first search traversal
A       B       D       E       F       C
```

****

Program No: 6

Write a program to implement Strassen's Matrix Multiplication of 2*2 Matrixes.

****

```
x = [[8, 5],
     [1, 7]]
y = [[3, 4],
     [2, 9]]
z = [[0, 0],
     [0, 0]]
print("The First Matrix is: ")
for i in x:
   print(i)
print("The Second Matrix is: ")
for i in y:
   print(i)

m1= (x[0][0] + x[1][1]) * (y[0][0] + y[1][1])
m2 = (x[1][0] + x[1][1]) * y[0][0]
m3 = x[0][0] * (y[0][1] - y[1][1])
m4 = x[1][1] * (y[1][0] - y[0][0])
m5 = (x[0][0] + x[0][1]) * y[1][1]
m6 = (x[1][0] - x[0][0]) * (y[0][0]+y[0][1])
m7 = (x[0][1] - x[1][1]) * (y[1][0]+y[1][1])
z[0][0] = m1 + m4- m5 + m7
z[0][1] = m3 + m5
z[1][0] = m2 + m4
z[1][1] = m1 - m2 + m3 + m6

print("The Product of Two Matrix is: ")
for i in z:
   print(i)
```

OUTPUT:

```
The First Matrix is:
[8, 5]
[1, 7]
The Second Matrix is:
[3, 4]
[2, 9]
The Product of Two Matrix is:
[34, 77]
[17, 67]
```

****

Program No: 7

Design and implement in to find a subset of a given set S ={Sl, S2,.....,Sn} of n positive integers whose SUM is equal to a given positive integer d.

For example, if S={1, 2, 5, 6, 8} and d= 9, there are two solutions {1,2,6} and {1,8}.

Display a suitable message, if the given problem instance doesn't have a solution.

****

```
def subset_sum(arr, target, index, current_subset, current_sum):
    global found

    if current_sum == target:
        print(current_subset)
        found = True
        return

    if index == len(arr) or current_sum > target:
        return

    subset_sum(arr, target, index + 1,
        current_subset + [arr[index]],
        current_sum + arr[index])

    subset_sum(arr, target, index + 1,
        current_subset,
        current_sum)

S = set(map(int, input("Enter the elements (separated by spaces):").split()))
d = int(input("Enter the target sum: "))
found = False
print("Subsets with sum", d, "are:")
subset_sum(list(S), d, 0, [], 0)

if not found:
    print("No subset exists with the given sum.")
```

OUTPUT 1:

```
Enter the elements (separated by spaces):1 2 3 4 5 6
Enter the target sum: 8
Subsets with sum 8 are:
[1, 2, 5]
[1, 3, 4]
[2, 6]
[3, 5]
```

OUTPUT 2:

```
Enter the elements (separated by spaces):1 2 3
Enter the target sum: 7
Subsets with sum 7 are:
No subset exists with the given sum.
```

PART-B

****

Program No: 1

Sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort.

****

```
import random
import time
def merge_sort(arr):
    #if the array has 1 or 0 elements, it is already sorted
    if len(arr) <= 1:
        return arr
    # Find the middle point and divide the array into two halves
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])
    # Merge the sorted halves
    return merge(left_half, right_half)
def merge(left, right):
    sorted_array = []
    i = j = 0
    # Compare elements from both halves and add the smaller one to the result
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            sorted_array.append(left[i])
            i += 1
        else:
            sorted_array.append(right[j])
            j += 1
    # Append any remaining elements (if one half is longer than the other)
    sorted_array.extend(left[i:])
    sorted_array.extend(right[j:])
    return sorted_array
#n = int(input("Enter the number of elements: "))
arr = []
for i in range(5005):
    n=random.randrange(5000)
    arr.append(n)
print('The Array Element Before sorting:')
print(arr)
start=time.time()
print("\nBefore Sorting Time :",start)
msort=merge_sort(arr)
end=time.time()
print('\nThe Array Element after sorting:')
print(msort)
print("\nAfter Sorting Time :",end)
dif=end-start
print("\nTime Taken :",dif)
```

OUTPUT :

```
The Array Element Before sorting:
Squeezed text (362 lines).

Before Sorting Time : 1791872582.0720174

The Array Element after sorting:
Squeezed text (362 lines).

After Sorting Time : 1791872582.1172085

Time Taken : 0.045191049575805664
```

****

Program No: 2

Write a program to perform Knapsack Problem.

****

```
def knapsack(n, capacity, weights, prices):
    # Base condition
    if n == 0 or capacity == 0:
        return 0
    # If the weight of current item is greater than capacity,
    # we cannot include it
    if weights[n - 1] > capacity:
        return knapsack(n - 1, capacity, weights, prices)
    else:
        # Option 1: Include the current item
        include = prices[n - 1] + knapsack(
            n - 1,
            capacity - weights[n - 1],
            weights,
            prices
        )
        # Option 2: Do not include the current item
        exclude = knapsack(
            n - 1,
            capacity,
            weights,
            prices
        )
        # Choose the option with maximum profit
        return max(include, exclude)
weights = [7, 3, 4, 5]
prices = [42, 12, 40, 25]
n = 4
capacity = 10
print("Knapsack Problem Solution")
print("Weights =", weights)
print("Prices  =", prices)
print("Capacity =", capacity)
print("Optimal Solution =", knapsack(n, capacity, weights, prices))
```

OUTPUT :

```
Knapsack Problem Solution
Weights = [7, 3, 4, 5]
Prices  = [42, 12, 40, 25]
Capacity = 10
Optimal Solution = 65
```

****

Program No: 3

Write a program to perform Travelling Salesman Problem

****

```
import itertools

def totaldistance(route, distance_matrix):
    distance = 0

    for i in range(len(route) - 1):
        distance += distance_matrix[route[i]][route[i + 1]]

     distance += distance_matrix[route[-1]][route[0]]

    return distance

def tsp(distance_matrix):
    n = len(distance_matrix)
    vertices = list(range(n))
    min_distance = float('inf')
    optimal_route = []

    for permutation in itertools.permutations(vertices[1:]):
        route = [0] + list(permutation)

        current_distance = totaldistance(route, distance_matrix)

        if current_distance < min_distance:
            min_distance = current_distance
            optimal_route = route

    return optimal_route, min_distance

distance_matrix = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

route, distance = tsp(distance_matrix)

print("Optimal route:", route + [route[0]])
print("Minimum distance:", distance)
```

OUTPUT :

```
Optimal route: [0, 1, 3, 2, 0]
Minimum distance: 80
```

****

Program No: 4

Write program to generate the Huffman code for the given character and probabilities.

Character	A	B	C	D	E
Probability	0.1	0.1	0.2	0.2	0.4

****

```
class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None


def build_huffman_tree(char_data):

    # Create a node for each character
    nodes = [Node(char, freq) for char, freq in char_data.items()]

    # Continue until only one node remains
    while len(nodes) > 1:

        # Sort in descending order
        # Smallest frequencies will be at the end
        nodes.sort(key=lambda x: x.freq, reverse=True)

        # Take the two smallest nodes
        node1 = nodes.pop()
        node2 = nodes.pop()

        # Create a new node by combining them
        merged = Node(None, round(node1.freq + node2.freq, 5))

        # Make the two nodes children
        merged.left = node1
        merged.right = node2

        # Add the new node back to the list
        nodes.append(merged)

    # The remaining node is the root
    return nodes[0]


def generate_codes(root, current_code="", codes=None):

    if codes is None:
```

```
        codes = {}

    if root is not None:

        # If it is a character node
        if root.char is not None:
            codes[root.char] = current_code

        # Left = 0
        generate_codes(root.left, current_code + "0", codes)

        # Right = 1
        generate_codes(root.right, current_code + "1", codes)

    return codes


# Input data
char_data = {
    "A": 0.1,
    "B": 0.1,
    "C": 0.2,
    "D": 0.2,
    "E": 0.4
}


# Build Huffman tree
root = build_huffman_tree(char_data)

# Generate Huffman codes
huffman_codes = generate_codes(root)


# Display the result
print(f"{'Character':<10} | {'Probability':<12} | {'Huffman Code'}")
print("-" * 40)

for char, data in char_data.items():
    print(f"{char:<10} | {data:<12} | {huffman_codes[char]}")
```

OUTPUT :

```
Character   | Probability   | Huffman Code
-----------------------------------------
A           | 0.1           | 1101
B           | 0.1           | 1100
C           | 0.2           | 10
D           | 0.2           | 111
E           | 0.4           | 0
```

****

Program No: 5

Write a program that implements Prim's algorithm to generate minimum cost spanning Tree.

****

```
INF = 9999999
N = 5
sum = 0

G = [
   [0, 19, 5, 0, 0],
   [19, 0, 5, 9, 2],
   [5, 5, 0, 1, 6],
   [0, 9, 1, 0, 1],
   [0, 2, 6, 1, 0]
]

Visited = [0, 0, 0, 0, 0]

V = 0

# Start from vertex 0
Visited[0] = True

print("\nWeight Matrix of a Given Graph")

for i in G:
   print(i)

print("\nThe minimum cost spanning Tree by Prim's Algorithm")
print("\nEdge \t:\t Weight\n")

while V < N - 1:

   minimum = INF
   a = 0
   b = 0

   # Find the minimum edge from visited
   # vertex to an unvisited vertex
   for m in range(N):

      if Visited[m]:

         for n in range(N):

            if (not Visited[n]) and G[m][n]:

               if minimum > G[m][n]:
                  minimum = G[m][n]
```

```
                    a = m
                    b = n

    print(str(a) + "-" + str(b) + "\t:\t" + str(G[a][b]))

    sum = sum + int(G[a][b])

    Visited[b] = True

    V += 1

print("\nTotal Cost = ", sum)
```

OUTPUT :

```
Weight Matrix of a Given Graph
[0, 19, 5, 0, 0]
[19, 0, 5, 9, 2]
[5, 5, 0, 1, 6]
[0, 9, 1, 0, 1]
[0, 2, 6, 1, 0]

The minimum cost spanning Tree by Prim's Algorithm

Edge    :        Weight

0-2     :       5
2-3     :       1
3-4     :       1
4-1     :       2

Total Cost =  9
```

****

Program No: 6

Write a program that implements Kruskal's algorithm to generate minimum cost spanning tree.

****

```
def find(parent, node):
    if parent[node] != node:
        parent[node] = find(parent, parent[node])
    return parent[node]


# Union function using rank
def union(parent, rank, u, v):
    root_u = find(parent, u)
    root_v = find(parent, v)

    if root_u == root_v:
        return False

    if rank[root_u] < rank[root_v]:
        parent[root_u] = root_v
    elif rank[root_u] > rank[root_v]:
        parent[root_v] = root_u
    else:
        parent[root_v] = root_u
        rank[root_u] += 1

    return True


# Kruskal's algorithm
def kruskal(n, edges):
    # Sort edges based on weight
    edges.sort()

    parent = [i for i in range(n)]
    rank = [0] * n
    mst = []
    mst_cost = 0

    for weight, u, v in edges:
        if union(parent, rank, u, v):
            mst.append((u, v, weight))
            mst_cost += weight

    return mst, mst_cost
```

```
n = 6

# List of edges: (weight, u, v)
edges = [
    (4, 0, 1),
    (4, 0, 2),
    (2, 1, 2),
    (5, 1, 3),
    (1, 2, 3),
    (3, 2, 4),
    (6, 3, 4),
    (7, 3, 5),
    (8, 4, 5)
]

mst, cost = kruskal(n, edges)

print("*****Kruskal's Algorithm*****\n")
print("Edges in the Minimum Spanning Tree:")

for u, v, weight in mst:
    print(f"{u} -- {v} == {weight}")

print("\nTotal cost of MST: ", cost)
```

OUTPUT :

```
*****Kruskal's Algorithm*****

Edges in the Minimum Spanning Tree:
2 -- 3 == 1
1 -- 2 == 2
2 -- 4 == 3
0 -- 1 == 4
3 -- 5 == 7

Total cost of MST:  17
```

****

Program No: 7

Write program for solving N-queens problem, by taking value of N as input.

****

```
def solve_n_queens(n):
    col = set()
    pos_diag = set()  # (r + c)
    neg_diag = set()  # (r - c)

    board = [["."] * n for _ in range(n)]
    solutions = []

    def backtrack(r):
        if r == n:
            # Save a copy of the current board configuration
            solutions.append(["".join(row) for row in board])
            return

        for c in range(n):
            # Check if the column or diagonals are already under attack
            if c in col or (r + c) in pos_diag or (r - c) in neg_diag:
                continue

            # Place the queen
            col.add(c)
            pos_diag.add(r + c)
            neg_diag.add(r - c)
            board[r][c] = "Q"

            # Recurse to place the next queen in the next row
            backtrack(r + 1)

            # Remove the queen (backtrack)
            col.remove(c)
            pos_diag.remove(r + c)
            neg_diag.remove(r - c)
            board[r][c] = "."

    backtrack(0)
    return solutions


if __name__ == "__main__":
    try:
        n = int(input("Enter the number of queens: "))

        if n <= 0:
            print("Please enter a positive integer greater than 0.")
        else:
            sols = solve_n_queens(n)
```

```
        print(f"\nTotal solutions for {n} Queens = {len(sols)}\n")

        for i, sol in enumerate(sols, 1):
            print(f"Solution {i}:")
            for row in sol:
                print(row)
            print()

    except ValueError:
        print("Invalid input! Please enter an integer.")
```

OUTPUT :

```
Enter the number of queens: 4

Total solutions for 4 Queens = 2

Solution 1:
.Q..
...Q
Q...
..Q.

Solution 2:
..Q.
Q...
...Q
.Q..
```